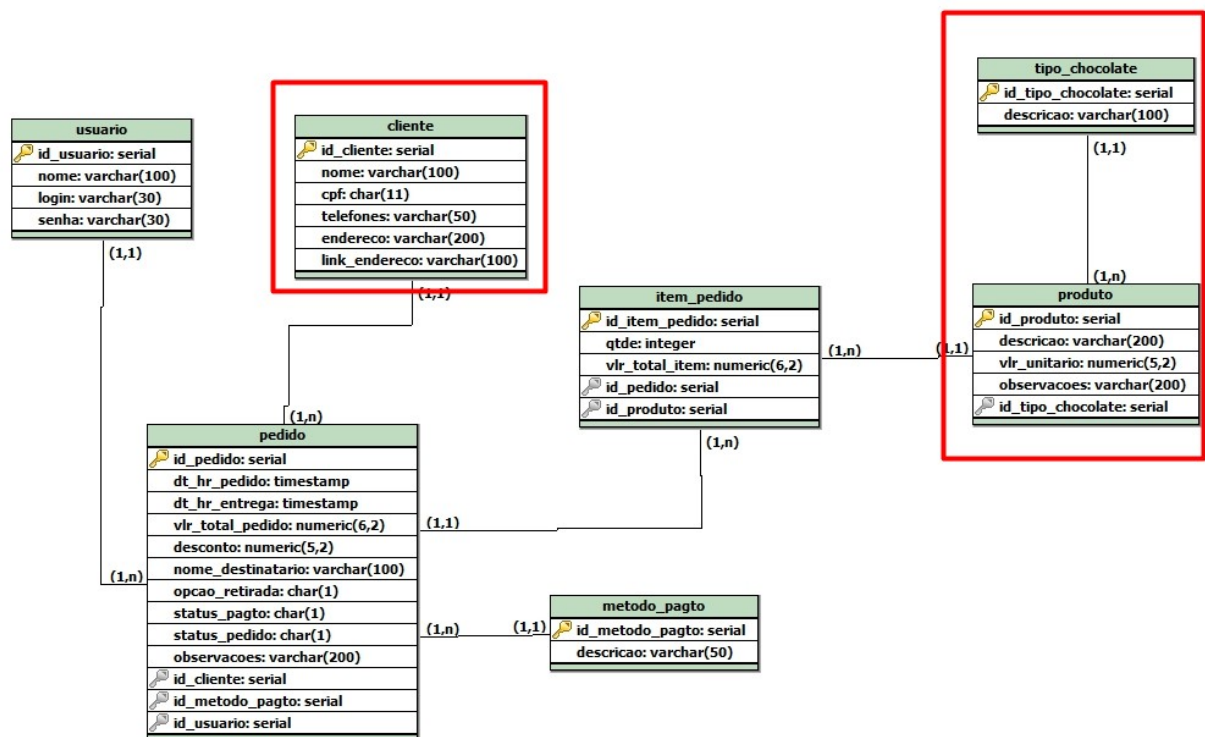


Telas Eleitas

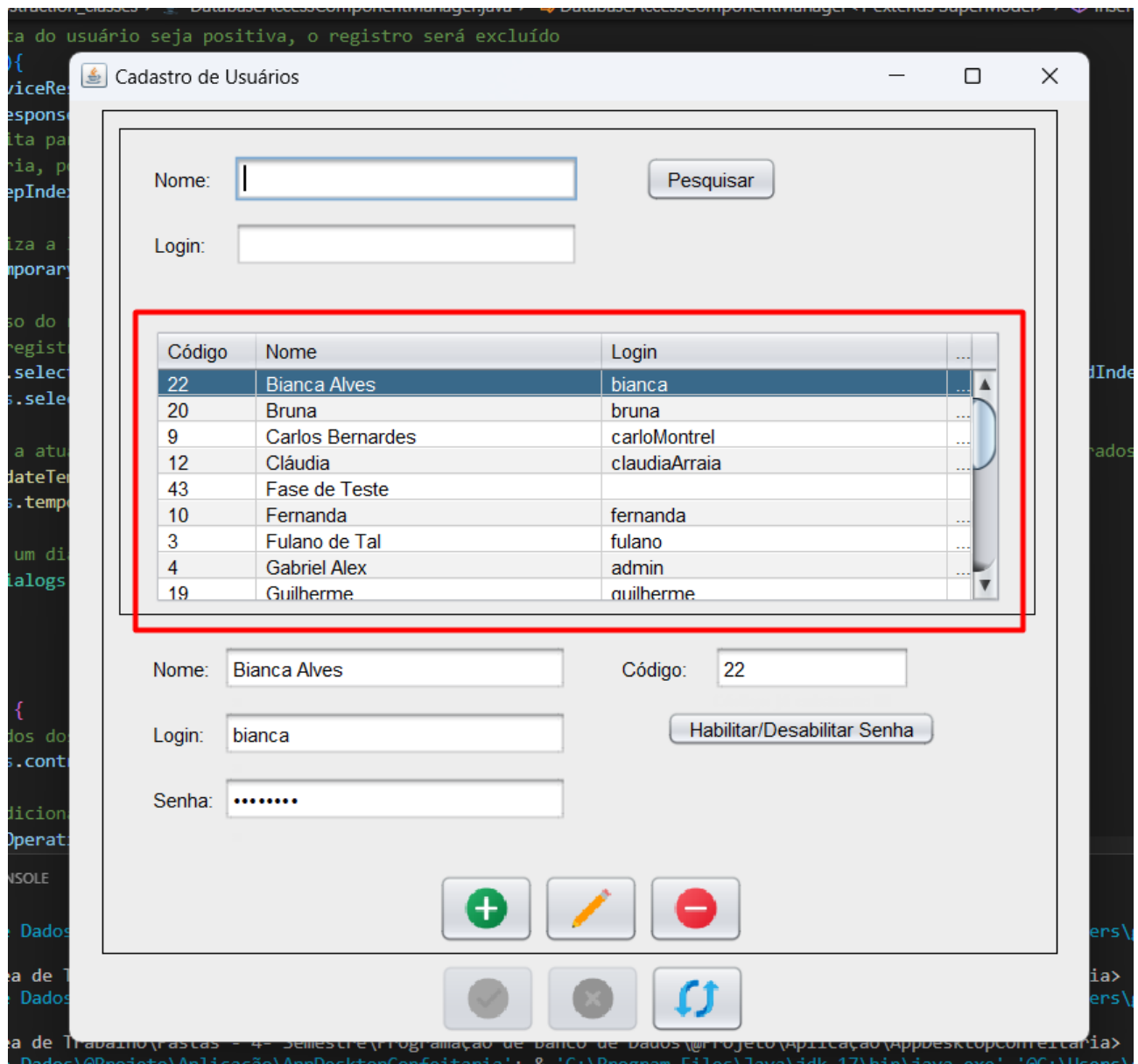
As telas eleitas até o momento para o desenvolvimento final do sistema foram a do cliente e a produto, contemplando a do produto a tabela “tipo_chocolate”. Após a imagem abaixo do DER das tabelas do BD indicando as que foram eleitas para o desenvolvimento, haverá uma sequência de explicações sobre o CRUD da tabela “usuario”(o primeiro a ser desenvolvido) com o objetivo exemplificar o funcionamento dos demais CRUDS, visto que todos seguem o exato mesmo padrão, com algumas pequenas diferenças como em situações onde há a interação com mais de uma tabela em uma mesma tela. Depois disso, haverá uma explanação do funcionamento básico das telas eleitas.



Tela Modelo: CRUD Usuário

Select

Ao selecionar os dados no banco, quem é responsável por exibi-los é a JTable(tabela do java swing):



Ela possui campos para que se possa filtrar a pesquisa. Porém, por padrão, a pesquisa inicial vem como total, sem nenhuma condição. É o mesmo método usado para fazer a pesquisa parcial, basta passar os parâmetros como nulos que a classe UsuarioDAO cuidará do resto, veja a seguir:

```
public List<Usuario> partialSearch(String nome, String login) {
    PreparedStatement statement = null;
    ResultSet resultSet = null;

    /*Verifica se os parâmetros foram fornecidos e se não são vazios, ou seja,
    confere se o usuário deseja realizar a pesquisa por nome e/ou login*/
    boolean selectByName = nome != null && !nome.isEmpty();
    boolean selectByLogin = login != null && !login.isEmpty();

    //Cria a query
    String sql =
        "SELECT *"+
        "FROM usuario " +
        "WHERE 1 = 1" +
        (selectByName ? "AND unaccent(nome) ILIKE unaccent(?)" : "") + //Se o usuário deseja pesquisar por nome, adiciona a condição à query
        (selectByLogin ? "AND unaccent(login) ILIKE unaccent(?)" : "") + //Se o usuário deseja pesquisar por login, adiciona a condição à query
        "ORDER BY nome, login";

    /*Define o padrão de pesquisa em relação aos parâmetros fornecidos pelo usuário*/
    nome = "%" + nome + "%";
    login = "%" + login + "%";

    //Define uma variável para armazenar o índice do parâmetro a ser configurado
    int paramIndexCount = 1;
    try{
        //Define o PreparedStatement com o SQL
        statement = DBConnection.getConnection().prepareStatement(sql);

        //Configura os parâmetros necessários
        if(selectByName) {
            statement.setString(paramIndexCount, nome);
            paramIndexCount++; //Incrementa o índice do parâmetro para que o próximo seja configurado corretamente
        }
        if(selectByLogin) {
            statement.setString(paramIndexCount, login);
        }

        //Armazena o SQL para haver referência da última consulta realizada pelo DAO
        this.lastSqlPartialSearch = statement.toString();

        //Obtém o ResultSet executando a query
        resultSet = statement.executeQuery();

        return this.resultSetToList(resultSet);
    } catch (SQLException e) {
        e.printStackTrace();
        return null;
    } finally {
        //Fecha a conexão com o banco de dados e os recursos criados a partir dela
        DBConnection.closeConnection(statement, resultSet);
    }
}
```

Por tanto, antes de exibir a tela, a View solicitará uma pesquisa total a Service que por sua vez acionará o método da DAO mostrado acima, passando parâmetros nulos. A DAO retornará o resultado para a Service, que por sua vez retornará para a View.

Para configurar a JTable para exibir os dados obtidos do banco, o seguinte código é aplicado:

```
private void displayDataInTable(List<T> tList) {
    //Limpando a tabela
    this.tableModel.setNumRows(rowCount:0);

    if(tList != null && tList.size() > 0) {
        //Adicionando os dados na tabela
        for(T tObject : tList) {
            try {
                this.tableModel.addRow(this.controller.modelToTableRow(tObject));
            } catch (Throwable e) {
                e.printStackTrace();
            }
        }
    }
}
```

Porém, esse método é de uso geral, para qualquer tabela da aplicação, cada tela tem que implementar um mapeamento do modelo do objeto para o formato da sua própria linha da tabela. A do usuário é o seguinte:

```
@Override
public Object[] modelToTableRow(Usuario usuario){
    return new Object[]{
        usuario.getID(),
        usuario.getNome(),
        usuario.getLogin(),
        usuario.getSenha()
    };
}
```

Toda a mesma lógica apresentada também é usada nas pesquisas parciais, com as únicas diferenças de que:

- O botão pesquisar da View aciona a pesquisa.
- Ao ser acionado por clique, os dados dos campos de pesquisa são capturados e repassados como argumentos para a Service, que por sua vez os usará para fazer a pesquisa parcial por meio do método da DAO descrito anteriormente.

Insert

Quando o botão de cadastro de registro é acionado as seguintes operações ocorrem na tela:

- Uma linha é adicionada na tabela indicando que um novo registro está sendo inserido.
- A tabela é desativada(para que o usuário não troque o registro selecionado no meio da edição).
- Os botões de edição(insert, update e delete) são desativados.
- Os botões de confirmação de edição(post e cancel) e o de atualização(refresh) são ativados.

The screenshot shows a web application window titled "Cadastro de Usuários". It contains a search section with "Nome:" and "Login:" input fields and a "Pesquisar" button. Below is a table of users. The table has columns "Código", "Nome", and "Login". The row for "José" (Código 11) is highlighted. Below the table are input fields for "Nome:", "Login:", and "Senha:", each with a red error message "Campo obrigatório !!!". There is also a "Código:" input field and a "Habilitar/Desabilitar Senha" button. At the bottom, there are three buttons: a green checkmark, a red X, and a blue refresh icon. The "Insere" button, which is a grey circle with a plus sign, is highlighted with a red box.

Código	Nome	Login
50	João Alves	joaoAlves21
48	João Mario	joaomario
70	João Paulo	jp007400
11	José	jose
69	Leia Oliveira	leiaOLiviera
14	Leticia	leticia
15	Lucas	lucas
13	Luiz Antonio	luizAMarques

Caso o usuário cancele a inserção, todos os dados inseridos nos campos serão limpos e a linha que havia sido adicionado na tabela será apagada. Caso ele deseje inserir os dados, apertando o botão salvar(post), ocorrem as seguintes operações, veja as imagens abaixo:

The interface displays a table of users and a form for adding a new user. The table has columns for Código, Nome, and Login. The form includes fields for Nome, Código, Login, and Senha, along with a 'Habilitar/Desabilitar Senha' button. Below the form are three buttons: a green checkmark (highlighted with a red box), a red X, and a refresh icon. A 'Salvar' label is positioned below the green checkmark button.

Código	Nome	Login
50	João Alves	joaoAlves21
48	João Mario	joamario
70	João Paulo	jp007400
11	José	jose
69	Leia Oliveira	leiaOLiviera
14	Leticia	leticia
15	Lucas	lucas
13	Luiz Antonio	luizAMarques

Nome: Código:

Login:

Senha:

Salvar

The interface shows the same table and form after the user has been saved. The 'Código' field now contains '220', and the new user 'Ana Maria Silva' is added to the table. The buttons at the bottom are now a green plus, a pencil, and a red minus.

Código	Nome	Login
50	João Alves	joaoAlves21
48	João Mario	joamario
70	João Paulo	jp007400
11	José	jose
220	Ana Maria Silva	anamaria
69	Leia Oliveira	leiaOLiviera
14	Leticia	leticia
15	Lucas	lucas
13	Luiz Antonio	luizAMarques

Nome: Código:

Login:

Senha:

Em código, quando o salvar é clicado, os dados dos campos são transformados em um objeto da classe usuário e o seguinte método da Service é acionado:

```
@Override
public boolean insert(Usuario usuario) {
    //Valida os dados do usuário
    if(this.validateDataInsert(usuario)) {
        /*Caso os dados sejam válidos, solicita ao DAO a inserção
        do usuário no banco de dados já retornando o resultado*/
        return this.usuarioDAO.insert(usuario);
    } else {
        return false;
    }
}
```

Na DAO, ocorre o seguinte:

```
public boolean insert(Usuario usuario) {
    PreparedStatement statement = null;
    try {
        //Verifica se o ID do usuário foi fornecido
        if(usuario.getID() == null) {
            //Se não foi, realiza a inserção sem o ID
            this.insertWithoutId(usuario, statement);
        } else {
            //Se foi, realiza a inserção com o ID
            this.insertWithId(usuario, statement);
        }
        //Se a inserção foi realizada com sucesso, retorna true
        return true;
    } catch (SQLException e) {
        e.printStackTrace();
        //Se a inserção não foi realizada com sucesso, retorna false
        return false;
    }
}

/*Método para inserir um usuário sem o ID. O método é privado pois as solicitações de inserção
devem ser feitas através do método insert(Usuario usuario) para que o ID seja verificado por ele*/
private void insertWithoutId(Usuario usuario, PreparedStatement statement) throws SQLException {
    //Cria a query
    String sql = "INSERT INTO usuario(nome, login, senha) " +
        "VALUES(?, ?, ?) RETURNING id_usuario";

    //Define o PreparedStatement com o SQL, em seguida, configura os parâmetros necessários
    statement = DBConnection.getConnection().prepareStatement(sql);
    statement.setString(parameterIndex:1, usuario.getNome());
    statement.setString(parameterIndex:2, usuario.getLogin());
    statement.setString(parameterIndex:3, usuario.getSenha());

    //Executa a query e obtém o ResultSet(que deverá conter o ID do usuário retornado pela inserção)
    ResultSet resultSet = statement.executeQuery();
    if(resultSet.next()) {
        //Se a inserção foi realizada com sucesso, define o ID do usuário
        usuario.setID(resultSet.getLong(columnLabel:"id_usuario"));
    }
    //Fecha a conexão com o banco de dados e os recursos criados a partir dela
    DBConnection.closeConnection(statement);
}
```

```

/*Método para inserir um usuário com o ID. O método é privado pois as solicitações de inserção
devem ser feitas através do método insert(Usuario usuario) para que o ID seja verificado por ele*/
private void insertWithId(Usuario usuario, PreparedStatement statement) throws SQLException {
    //Cria a query
    String sql = "INSERT INTO usuario(id_usuario, nome, login, senha) " +
        "VALUES(?, ?, ?, ?)";

    //Define o PreparedStatement com o SQL, em seguida, configura os parâmetros necessários
    statement = DBConnection.getConnection().prepareStatement(sql);
    statement.setLong(parameterIndex:1, usuario.getID());
    statement.setString(parameterIndex:2, usuario.getNome());
    statement.setString(parameterIndex:3, usuario.getLogin());
    statement.setString(parameterIndex:4, usuario.getSenha());

    //Executa a query
    statement.executeUpdate();

    //Fecha a conexão com o banco de dados e os recursos criados a partir dela
    DBConnection.closeConnection(statement);
}

```

Update

No update, basta que o usuário aperte o botão de edição(o lápis amarelo) ou que alterar o valor de algum dos campos que o sistema saberá que o registro está sendo editado. As mesmas operações visuais do insert ocorrem: Uma linha é adicionada na tabela, a tabela é desativada, os botões de edição são desativados e os botões de confirmação de edição e o de atualização são ativados.

Segue um exemplo completo:

The screenshot displays a user management interface. At the top, there is a table with columns 'Código', 'Nome', and 'Login'. The table contains several rows, with the row for 'Bianca Alves' (Código: 22) highlighted. Below the table, there is a form for editing a user. The form has fields for 'Nome', 'Login', 'Senha', and 'Código'. The 'Nome' field contains 'Bianca Alves Amorim', the 'Login' field contains 'biancaAmorim', and the 'Código' field contains '22'. There is a button labeled 'Habilitar/Desabilitar Senha'. At the bottom of the form, there are three buttons: a green checkmark, a red X, and a blue refresh icon.

Código	Nome	Login
220	Ana Maria Silva	anamaria
22	Bianca Alves	bianca
20	Bruna	bruna
9	Carlos Bernardes	carloMontrel
12	Cláudia	claudiaArraia
43	Fase de Teste	
10	Fernanda	fernanda
3	Fulano de Tal	fulano
4	Gabriel Alex	admin

Nome: Código:

Login:

Senha:

Buttons: [Add] [Edit] [Delete] [Confirm] [Cancel] [Refresh]

Se o salvar for clicado:

The screenshot shows a user management interface. At the top is a table with columns 'Código', 'Nome', and 'Login'. The table contains several rows, with the row for 'Bianca Alves Amorim' (Código: 22) highlighted in blue. A red rectangle highlights the row for 'Bianca Alves Amorim' in the table and the corresponding input fields in the form below. The form has fields for 'Nome' (containing 'Bianca Alves Amorim'), 'Código' (containing '22'), 'Login' (containing 'biancaAmorim'), and 'Senha' (containing '.....'). There is a button labeled 'Habilitar/Desabilitar Senha' next to the login field. Below the form are three buttons: a green '+' button, a yellow pencil icon button, and a red '-' button. At the bottom are three more buttons: a grey checkmark button, a grey 'X' button, and a blue circular arrow button.

Código	Nome	Login
220	Ana Maria Silva	anamaría
22	Bianca Alves Amorim	biancaAmorim
20	Bruna	bruna
9	Carlos Bernardes	carloMontrel
12	Cláudia	claudiaArraia
43	Fase de Teste	
10	Fernanda	fernanda
3	Fulano de Tal	fulano
4	Gabriel Alex	admin

Nome: Bianca Alves Amorim

Código: 22

Login: biancaAmorim

Senha:

Habilitar/Desabilitar Senha

Em código, a lógica é parecida com a operação do insert. Os dados dos campos são transformados em um objeto da classe usuário, o objeto do usuário contendo todos os dados originais também é capturado e o seguinte método da Service é acionado:

```
@Override
public boolean update(Usuario usuario, Usuario usuarioOriginal) {
    //Valida os dados do usuário
    if(this.validateDataUpdate(usuario, usuarioOriginal)) {
        /*Caso os dados sejam válidos, solicita ao DAO a atualização
        do usuário no banco de dados já retornando o resultado*/
        return this.usuarioDAO.update(usuario, usuarioOriginal.getID());
    } else {
        return false;
    }
}
```

Na DAO, ocorre o seguinte:

```
public boolean update(Usuario usuario, Long originalID) {
    if(originalID == null) return false;
    PreparedStatement statement = null;

    //Cria a query
    String sql = "UPDATE usuario " +
        "SET id_usuario = ?, nome = ?, login = ?, senha = ? " +
        "WHERE id_usuario = ?";

    try {
        //Define o PreparedStatement com o SQL, em seguida, configura os parâmetros necessários
        statement = DBConnection.getConnection().prepareStatement(sql);
        statement.setLong(parameterIndex:1, usuario.getID());
        statement.setString(parameterIndex:2, usuario.getNome());
        statement.setString(parameterIndex:3, usuario.getLogin());
        statement.setString(parameterIndex:4, usuario.getSenha());
        statement.setLong(parameterIndex:5, originalID);

        //Executa a query
        statement.executeUpdate();

        //Se a atualização foi realizada com sucesso, retorna true
        return true;
    } catch (SQLException e) {
        e.printStackTrace();
        //Se a atualização não foi realizada com sucesso, retorna false
        return false;
    } finally {
        //Fecha a conexão com o banco de dados e os recursos criados a partir dela
        DBConnection.closeConnection(statement);
    }
}
```

Delete

Por último, a operação delete. Para ela ser acionada, com um registro na tabela já selecionado, o botão de excluir deve ser clicado. Após isso, um dialog de confirmação de exclusão de registro será exibido na tela. Segue o exemplo:

The screenshot shows a user management interface. At the top, there is a table with columns 'Código', 'Nome', and 'Login'. The table contains several records, with the record having 'Código' 22 (Bianca Alves Amorim) selected. Below the table, there are input fields for 'Nome' (Bianca Alves Amorim), 'Login' (biancaAmorim), and 'Senha' (masked with dots). A 'Habilitar/Desabilitar Senha' button is next to the login field. A modal dialog box titled 'Confirmação de exclusão' is displayed in the foreground. It contains a question mark icon and the text 'Apagar o registro de código 22?'. At the bottom of the dialog are 'Ok' and 'Cancelar' buttons.

Código	Nome	Login
220	Ana Maria Silva	anamaria
22	Bianca Alves Amorim	biancaAmorim
20	Bruna	
9	Carlos Bern	
12	Cláudia	
43	Fase de Te	
10	Fernanda	
3	Fulano de T	
4	Gabriel Alex	

Nome: Bianca Alves Amorim Código: 22

Login: biancaAmorim Habilitar/Desabilitar Senha

Senha:

Confirmação de exclusão

Apagar o registro de código 22?

Ok Cancelar

Caso o usuário clique em “Cancelar” nada acontecerá, caso aperte em “Ok” o registro será apagado do banco de dados e em seguida apagado da tabela.

Em código, caso o usuário tenha apertado em “Ok”, o método da Service mostrado abaixo é chamado:

```
@Override
public boolean delete(Usuario usuario) {
    //Solicita ao DAO a exclusão do usuário no banco de dados já retornando o resultado
    return this.usuarioDAO.delete(usuario.getID());
}
```

Na DAO:

```
public boolean delete(Long idUsuario) {
    if(idUsuario == null) return false;
    PreparedStatement statement = null;

    //Cria a query
    String sql = "DELETE FROM usuario " +
                "WHERE id_usuario = ?";

    try {
        //Define o PreparedStatement com o SQL, em seguida, configura os parâmetros necessários
        statement = DBConnection.getConnection().prepareStatement(sql);
        statement.setLong(parameterIndex:1, idUsuario);

        //Executa a query
        statement.executeUpdate();

        //Se a exclusão foi realizada com sucesso, retorna true
        return true;
    } catch (SQLException e) {
        e.printStackTrace();
        //Se a exclusão não foi realizada com sucesso, retorna false
        return false;
    } finally {
        //Fecha a conexão com o banco de dados e os recursos criados a partir dela
        DBConnection.closeConnection(statement);
    }
}
```

Telas Eleitas: CRUD Cliente

A implementação do CRUD da tabela “cliente” segue a exatamente a mesma lógica de todas as ações(consulta, inserção, atualização, exclusão) do CRUD do usuário. Segue duas imagens da tela:

The screenshot shows the 'Cadastro de Clientes' application window. At the top, there is a search form with fields for 'Nome' and 'CPF', and a 'Pesquisar' button. Below the search form is a table listing clients. The table has columns: Código, Nome, CPF, Telefones, Endereço, and Link E. The table contains 10 rows of data. Below the table is a form for editing a client, with fields for 'Nome', 'CPF', 'Telefones', 'Endereço', and 'Link End.'. At the bottom of the form are three buttons: a green '+' button, a yellow pencil button, and a red '-' button. Below these buttons are three circular icons: a grey circle, a grey circle with an 'X', and a blue circular arrow.

Código	Nome	CPF	Telefones	Endereço	Link E
5	Ana Paula da Silva	45678901234	(31) 7777-7777	Rua das Estrelas, 789, Belo Horizon	https://m
11	André da Silva	45678901234	(95) 1111-1111	Rua das Ondas, 456, Belém/PA	https://m
7	Carlos Alberto da Silva	89012345678	(51) 5555-5555	Rua dos Sol, 890, Porto Alegre/RS	https://m
12	Clara da Silva	56789012345	(10) 0000-0000	Rua das Águas, 567, Macapá/AP	https://m
8	Fabiana da Silva	01234567890	(62) 4444-4444	Rua das Nuvens, 012, Goiânia/GO	https://m
1	João da Silva	12345678900	(11) 9999-9999	Rua das Flores, 123, São Paulo/SP	https://m
3	João da Silva	12345678900	(11) 9999-9999	Rua das Flores, 123, São Paulo/SP	https://m
9	José Carlos da Silva	23456789012	(73) 3333-3333	Rua dos Ventos, 234, Salvador/BA	https://m

The screenshot shows the 'Cadastro de Clientes' application window. The search form at the top has 'Nome' filled with 'jo' and 'CPF' empty. The 'Pesquisar' button is highlighted. Below the search form, the table shows the results of the search, displaying only the rows for 'João da Silva' (Código 1 and 3) and 'José Carlos da Silva' (Código 9). Below the table is the same edit form as in the first screenshot, but the fields are populated with the data for 'João da Silva': 'Nome: João da Silva', 'CPF: 12345678900', 'Telefones: (11) 9999-9999', 'Endereço: Rua das Flores, 123, São Paulo/SP', and 'Link End.: https://maps.google.com/?q=Rua+das+Flores,+123,+São+Paulo,+SP'. The buttons and icons at the bottom are the same as in the first screenshot.

Código	Nome	CPF	Telefones	Endereço	Link End
1	João da Silva	12345678900	(11) 9999-9999	Rua das Flores, 123, São Paulo/SP	https://m
3	João da Silva	12345678900	(11) 9999-9999	Rua das Flores, 123, São Paulo/SP	https://m
9	José Carlos da Silva	23456789012	(73) 3333-3333	Rua dos Ventos, 234, Salvador/BA	https://m

Telas Eleitas: CRUD Produto

O mesmo serve para o CRUD do produto, com a exceção de que há um relacionamento 1xn nessa implementação, que seria da tabela “produto” com a “tipo_chocolate”. O componente visual responsável por mostrar os registros da tabela “tipo_chocolate” foi o “ComboBox”, segue uma imagem mostrando seu funcionamento:

The screenshot shows the 'Cadastro de Produtos' window. At the top, there is a search bar labeled 'Descrição:' with a 'Pesquisar' button. Below it is a dropdown menu for 'Tipo Choc.' currently set to 'Todos'. A table displays a list of products with columns: Código, Descrição, Tipo do Chocolate, Valor Unitário, and Observações. The table contains 12 rows of data. Below the table, there is a form for adding or editing a product. The 'Descrição:' field contains 'amedoim'. The 'Tipo Choc.' dropdown menu is open, showing options: Branco, Amargo, Ao Leite, Branco, and Meio Amargo. The 'Vir. Unit.' field contains '10.00'. At the bottom, there are buttons for adding (+), editing (pencil), and deleting (-) products, along with buttons for saving, canceling, and refreshing.

Código	Descrição	Tipo do Chocolate	Valor Unitário	Observações
28	amedoim	Branco	10.00	sACDSCASDCASD
29	Bolo de Abacaxi com chocolate	Meio Amargo	55.75	gxhnfdcfghidjfh
9	Bombom de chocolate ao leite	Amargo	9.00	Bombom de chocol
15	Bombom de chocolate ao leite diet	Amargo	10.00	Bombom de chocol
21	Bombom de chocolate ao leite vega...	Amargo	11.00	Bombom de chocol
6	Bombom de chocolate branco	Amargo	8.00	Bombom de chocol
3	Bombom de chocolate branco	Amargo	8.00	Bombom de chocol
12	Bombom de chocolate branco diet	Amargo	10.00	Bombom de chocol

Visão geral do CRUD:

The screenshot shows the 'Cadastro de Produtos' window. At the top, there is a search bar labeled 'Descrição:' with a 'Pesquisar' button. Below it is a dropdown menu for 'Tipo Choc.' currently set to 'Todos'. A table displays a list of products with columns: Código, Descrição, Tipo do Chocolate, Valor Unitário, and Observações. The table contains 12 rows of data. Below the table, there is a form for adding or editing a product. The 'Descrição:' field contains 'Bombom de chocolate ao leite'. The 'Tipo Choc.' dropdown menu is set to 'Amargo'. The 'Vir. Unit.' field contains '9.00'. The 'Observações:' field contains 'Bombom de chocolate ao leite com recheio de nozes'. At the bottom, there are buttons for adding (+), editing (pencil), and deleting (-) products, along with buttons for saving, canceling, and refreshing.

Código	Descrição	Tipo do Chocolate	Valor Unitário	Observações
9	Bombom de chocolate ao leite	Amargo	9.00	Bombom de chocol
15	Bombom de chocolate ao leite diet	Amargo	10.00	Bombom de chocol
21	Bombom de chocolate ao leite vega...	Amargo	11.00	Bombom de chocol
6	Bombom de chocolate branco	Amargo	8.00	Bombom de chocol
3	Bombom de chocolate branco	Amargo	8.00	Bombom de chocol
12	Bombom de chocolate branco diet	Amargo	10.00	Bombom de chocol
18	Bombom de chocolate branco vega...	Amargo	11.00	Bombom de chocol
1	Brigadeiro de chocolate ao leite	Ao Leite	10.00	Brigadeiro tradicion

Diferenças em relação ao CRUD modelo

– Select:

```
public List<Produto> partialSearch(String descricao, String tipoChocolate) {
    PreparedStatement statement = null;
    ResultSet resultSet = null;

    /*Verifica se os parâmetros foram fornecidos e se não são vazios, ou seja,
    confere se o usuário deseja realizar a pesquisa pela descrição do produto e/ou pelo tipo do chocolate*/
    boolean selectByDescricao = descricao != null && !descricao.isEmpty();
    boolean selectByTipoChocolate = tipoChocolate != null && !tipoChocolate.isEmpty();

    //Cria a query
    String sql = "SELECT p.*, tc.descricao as tipo_chocolate FROM produto p LEFT JOIN tipo_chocolate tc ON p.id_tipo_chocolate = tc.id_tipo_chocolate " +
        "WHERE 1 = 1" +
        (selectByDescricao ? "AND unaccent(p.descricao) ILIKE unaccent(?)" : "") + // Se o usuário deseja pesquisar pela descrição, adiciona a condição
        (selectByTipoChocolate ? "AND tc.descricao = ?" : "") + // Se o usuário deseja pesquisar pelo tipo do chocolate, adiciona a condição
        "ORDER BY p.descricao, tc.descricao, p.vlr_unitario";

    /*Define o padrão de pesquisa em relação aos parâmetros fornecidos pelo usuário*/
    descricao = "%" + descricao + "%";
}
```

Comentários: Precisei usar 'LEFT JOIN' para conseguir obter também os produtos que tinham 'tipo_chocolate' nulo, já que eu permiti essa possibilidade no meu sistema.

– ComboBox:

```
@Override
public void displayView() {
    this.produtoView.getCmbTipoChoc().removeAllItems();
    this.produtoView.getCmbTipoChocFiltro().removeAllItems();
    for (String value : this.produtoService.selectAllTipoChocolate()) {
        this.produtoView.getCmbTipoChoc().addItem(value);
        this.produtoView.getCmbTipoChocFiltro().addItem(value);
    }
    this.produtoView.getCmbTipoChocFiltro().addItem(allOptionsText);
    this.produtoView.getCmbTipoChocFiltro().setSelectedItem(allOptionsText);
    /*Cancela qualquer operação que possa estar sendo realizada, isso porque houve
    alterações acima em um dos fields, logo, o manager entende que estava havendo uma
    operação de atualização, o que não era o caso, eram apenas configurações iniciais*/
    this.produtoDBCManager.cancel();
    this.produtoView.setVisible(b:true);
}
```

Comentários: Essas instruções foram realizadas na controller da tela do produto, elas foram colocadas no método que exibe a tela do produto, antes da instrução que torna a tela visível. Os passos que realizei foram os seguintes:

1. Removi todos os itens dos ComboBox para caso houvesse algum e não se misturasse com os corretos.

2. Solicitei a service todas as opções de tipo de chocolate, o resultado é uma lista de string com as descrições de cada tipo de chocolate. A query usada na DAO é a seguinte:

```
public List<String> selectAllTipoChocolate() {  
    //Cria o PreparedStatement e o ResultSet  
    PreparedStatement statement = null;  
    ResultSet resultSet = null;  
  
    //Cria a query  
    String sql = "SELECT descricao " +  
                "FROM tipo_chocolate " +  
                "ORDER BY descricao";  
}
```

3. Por meio da lista recebida da service realizei um for each para pegar cada item da lista e adicionar como item dos dois ComboBox que mostram os tipos do chocolate na minha tela.
4. Por fim, no ComboBox do filtro, adicionei a opção "Todos" para que, caso o usuário queira, ele possa selecionar por todos os produtos, desconsiderando o tipo do chocolate.

